

Filière : 3^{ème} Année Ingénierie Informatique et Réseaux	Correction
	Travail pratique n°2
Module : Compétences Numériques Matière : Développement Web	Professeur : Y. JDIDOU

Scénario Étendu : Gestion Avancée de Tâches

Contexte : Vous êtes une équipe de développeurs travaillant sur une application de gestion de tâches collaborative. Votre client souhaite ajouter de nouvelles fonctionnalités pour améliorer l'expérience utilisateur. Ces fonctionnalités doivent être intégrées dans l'application existante. (Utiliser le fichier **GestionTaches.html**)

Nouvelles Fonctionnalités à Implémenter

1. Marquer une tâche comme terminée

- Ajoutez une case à cocher à chaque tâche pour indiquer qu'elle est terminée.
- Les tâches terminées doivent être affichées avec un style différent (par exemple, barré ou grisé).
- Implémentez une méthode pour basculer l'état d'une tâche (**completed: true/false**).

```
1 function toggleTaskCompletion(id) {
2     const task = tasks.find(t => t.id === id);
3     if (!task) return;
4
5     task.completed = !task.completed;
6     renderTasks();
7 }
```

Modification de **renderTasks** :

```
1 function renderTasks() {
2     const taskList = document.getElementById('taskList');
3     taskList.innerHTML = '<h2>Liste des Tâches</h2>';
4
5     tasks.forEach(task => {
6         const taskItem = document.createElement('div');
7         taskItem.className = 'task-item';
8         taskItem.style.textDecoration = task.completed ? 'line-through' :
'none';
9
10        taskItem.innerHTML = `
11            <input type="checkbox" ${task.completed ? 'checked' : ''}
onclick="toggleTaskCompletion(${task.id})">
12            <span>${task.title}</span>
13            <div>
14                <button
onclick="handleUpdateTask(${task.id})">Modifier</button>
15                <button
onclick="handleDeleteTask(${task.id})">Supprimer</button>
16            </div>
17        `;
18    });
19 }
```

```

18     taskList.appendChild(taskItem);
19     });
20 }

```

2. Filtrage des tâches

- Ajoutez des boutons ou un menu déroulant pour afficher :
 - Toutes les tâches.
 - Les tâches terminées.
 - Les tâches en cours.

```

1  function filterTasks(filter) {
2      const filteredTasks = tasks.filter(task => {
3          if (filter === 'completed') return task.completed;
4          if (filter === 'incomplete') return !task.completed;
5          return true; // Tous les filtres
6      });
7      renderFilteredTasks(filteredTasks);
8  }
9
10 function renderFilteredTasks(filteredTasks) {
11     const taskList = document.getElementById('taskList');
12     taskList.innerHTML = '<h2>Liste des Tâches</h2>';
13
14     filteredTasks.forEach(task => {
15         const taskItem = document.createElement('div');
16         taskItem.className = 'task-item';
17         taskItem.innerHTML = `
18             <span>${task.title}</span>
19             <div>
20                 <button
21                     onclick="handleUpdateTask(${task.id})">Modifier</button>
22                 <button
23                     onclick="handleDeleteTask(${task.id})">Supprimer</button>
24                 </div>
25             `;
26         taskList.appendChild(taskItem);
27     });
28 }

```

HTML associé :

```

1 <select id="taskFilter" onchange="filterTasks(this.value)">
2     <option value="all">Toutes les tâches</option>
3     <option value="completed">Terminées</option>
4     <option value="incomplete">En cours</option>
5 </select>

```

3. Recherche en temps réel

- Ajoutez une barre de recherche permettant de filtrer les tâches en fonction de leur titre.
- Utilisez un événement **input** pour afficher instantanément les résultats.

```
1 function searchTasks(query) {
2     const searchQuery = query.toLowerCase();
3     const filteredTasks = tasks.filter(task =>
task.title.toLowerCase().includes(searchQuery));
4     renderFilteredTasks(filteredTasks);
5 }
```

HTML associé :

```
1 <input type="text" id="searchBar" placeholder="Rechercher..."
oninput="searchTasks(this.value)">
```

4. Gestion des priorités

- Ajoutez une propriété **priority** à chaque tâche (ex : **faible**, **moyenne**, **élevée**).
- Permettez à l'utilisateur de définir la priorité lors de l'ajout ou la modification d'une tâche.
- Implémentez une fonction pour trier les tâches par priorité.

```
1 function sortTasksByPriority() {
2     tasks.sort((a, b) => {
3         const priorityOrder = { 'low': 1, 'medium': 2, 'high': 3 };
4         return priorityOrder[b.priority] - priorityOrder[a.priority];
5     });
6     renderTasks();
7 }
```

Modification lors de l'ajout de tâches :

```
1 async function handleAddTask() {
2     const taskTitle = document.getElementById('taskTitle').value;
3     const taskPriority = document.getElementById('taskPriority').value;
4
5     if (!taskTitle) {
6         alert("Le titre de la tâche est requis !");
7         return;
8     }
9
10    const newTask = { id: tasks.length + 1, title: taskTitle, priority:
taskPriority, completed: false };
11    try {
12        const task = await simulateApiCall(newTask);
13        tasks.push(task);
14        sortTasksByPriority();
15        renderTasks();
16        document.getElementById('taskTitle').value = ''; // Réinitialiser
le champ
17        alert("Tâche ajoutée avec succès !");
18    } catch (error) {
19        alert(error);
20    }
21 }
```

HTML associé :

```
1 <select id="taskPriority">
2   <option value="low">Faible</option>
3   <option value="medium">Moyenne</option>
4   <option value="high">Élevée</option>
5 </select>
```

5. Sauvegarde des tâches dans le stockage local

- Utilisez **localStorage** pour sauvegarder la liste des tâches afin de les récupérer après un rechargement de la page.
- Implémentez une méthode pour charger les tâches à partir du localStorage au démarrage de l'application.

```
1 function saveToLocalStorage() {
2   localStorage.setItem('tasks', JSON.stringify(tasks));
3 }
4
5 function loadFromLocalStorage() {
6   const savedTasks = JSON.parse(localStorage.getItem('tasks')) || [];
7   tasks = savedTasks;
8   renderTasks();
9 }
10
11 // Appeler au chargement
12 window.onload = loadFromLocalStorage;
```

6. Mode "Dark Mode"

- Ajoutez un bouton pour basculer entre un thème clair et un thème sombre.
- Utilisez JavaScript pour modifier dynamiquement le style du site.

```
1 function toggleDarkMode() {
2   document.body.classList.toggle('dark-mode');
3 }
```

CSS associé :

```
1 .dark-mode {
2   background-color: #121212;
3   color: #ffffff;
4 }
```

HTML associé :

```
1 <button onclick="toggleDarkMode()">Activer/Désactiver Mode Sombre</button>
```

7. Drag-and-Drop

- Implémentez une fonctionnalité de glisser-déposer pour réorganiser les tâches.
- Permettez à l'utilisateur de changer l'ordre des tâches en les déplaçant avec la souris.

HTML associé :

```
1 <ul id="taskList">
2   <!-- Les tâches sont générées dynamiquement -->
3   <li class="task-item" draggable="true">Tâche 1</li>
4   <li class="task-item" draggable="true">Tâche 2</li>
5   <li class="task-item" draggable="true">Tâche 3</li>
6 </ul>
```

CSS (pour visuels Drag-and-Drop) :

```
1 .task-item {
2   padding: 10px;
3   margin: 5px 0;
4   background-color: #f9f9f9;
5   border: 1px solid #ddd;
6   border-radius: 4px;
7   cursor: grab;
8 }
9
10 .task-item.dragging {
11   opacity: 0.5;
12   background-color: #d1e7dd;
13 }
14
15 .task-item.drag-over {
16   border: 2px dashed #4caf50;
17 }
```

JavaScript :

```
1 let draggedTask = null;
2
3 function enableDragAndDrop() {
4   const taskItems = document.querySelectorAll('.task-item');
5
6   taskItems.forEach(task => {
7     // Déclenché au début du drag
8     task.addEventListener('dragstart', (e) => {
9       draggedTask = task;
10      task.classList.add('dragging');
11    });
12
13    // Déclenché lorsque le drag se termine
14    task.addEventListener('dragend', () => {
15      draggedTask.classList.remove('dragging');
16      draggedTask = null;
17    });
18
19    // Permet le dragover
20    task.addEventListener('dragover', (e) => {
21      e.preventDefault(); // Nécessaire pour permettre le drop
22      task.classList.add('drag-over');
23    });
24
25    // Déclenché lorsqu'un élément est lâché
```

```

26     task.addEventListener('dragleave', () => {
27         task.classList.remove('drag-over');
28     });
29
30     task.addEventListener('drop', (e) => {
31         e.preventDefault();
32         task.classList.remove('drag-over');
33
34         // Réorganiser les tâches
35         const taskList = document.getElementById('taskList');
36         if (draggedTask && draggedTask !== task) {
37             const tasksArray = Array.from(taskList.children);
38             const draggedIndex = tasksArray.indexOf(draggedTask);
39             const targetIndex = tasksArray.indexOf(task);
40
41             // Réorganiser dans le DOM
42             if (draggedIndex < targetIndex) {
43                 taskList.insertBefore(draggedTask, task.nextSibling);
44             } else {
45                 taskList.insertBefore(draggedTask, task);
46             }
47
48             // Mettre à jour les données internes si nécessaire
49             updateTaskOrder();
50         }
51     });
52 });
53 }
54
55 // Appeler la fonction après avoir généré les tâches dynamiquement
56 enableDragAndDrop();
57
58 // Mettre à jour l'ordre des tâches dans les données internes
59 function updateTaskOrder() {
60     const taskItems = document.querySelectorAll('.task-item');
61     tasks = Array.from(taskItems).map((task, index) => {
62         return { ...tasks[index], title: task.textContent.trim() };
63     });
64     console.log('Nouvel ordre des tâches :', tasks);
65 }

```

8. Ajout de sous-tâches

- Ajoutez une fonctionnalité permettant de créer des sous-tâches associées à une tâche principale.
- Affichez les sous-tâches sous leur tâche principale, avec des options pour les marquer comme terminées ou les supprimer.

Modifier la structure des données

Les tâches doivent contenir une propriété **subtasks**, qui est un tableau de sous-tâches.

```
1 let tasks = [  
2   {  
3     id: 1,  
4     title: "Tâche principale 1",  
5     completed: false,  
6     subtasks: [  
7       { id: 1, title: "Sous-tâche 1.1", completed: false },  
8       { id: 2, title: "Sous-tâche 1.2", completed: true }  
9     ]  
10  },  
11  {  
12    id: 2,  
13    title: "Tâche principale 2",  
14    completed: false,  
15    subtasks: []  
16  }  
17 ];
```

Ajouter une interface utilisateur pour gérer les sous-tâches

- Ajouter un bouton "Ajouter une sous-tâche" sous chaque tâche principale.
- Afficher les sous-tâches sous forme de liste imbriquée.

```
1 <div id="taskList">  
2   <!-- Exemple d'affichage dynamique -->  
3   <div class="task-item">  
4     <span>Tâche principale 1</span>  
5     <button onclick="addSubtask(1)">Ajouter une sous-tâche</button>  
6     <ul>  
7       <li>Sous-tâche 1.1</li>  
8       <li>Sous-tâche 1.2</li>  
9     </ul>  
10  </div>  
11 </div>
```

Ajouter la logique pour gérer les sous-tâches

- Ajouter une sous-tâche

```
1 function addSubtask(taskId) {  
2   const subtaskTitle = prompt("Entrez le titre de la sous-tâche :");  
3   if (!subtaskTitle) return;  
4  
5   const task = tasks.find(t => t.id === taskId);  
6   if (!task) return;  
7  
8   const newSubtask = {  
9     id: task.subtasks.length + 1,
```

```

10         title: subtaskTitle,
11         completed: false
12     };
13
14     task.subtasks.push(newSubtask);
15     renderTasks();
16 }

```

- Marquer une sous-tâche comme terminée

```

1 function toggleSubtaskCompletion(taskId, subtaskId) {
2     const task = tasks.find(t => t.id === taskId);
3     if (!task) return;
4
5     const subtask = task.subtasks.find(st => st.id === subtaskId);
6     if (!subtask) return;
7
8     subtask.completed = !subtask.completed;
9     renderTasks();
10 }

```

- Supprimer une sous-tâche

```

1 function deleteSubtask(taskId, subtaskId) {
2     const task = tasks.find(t => t.id === taskId);
3     if (!task) return;
4
5     task.subtasks = task.subtasks.filter(st => st.id !== subtaskId);
6     renderTasks();
7 }

```

Affichage dynamique des sous-tâches

Modifier la fonction **renderTasks** pour afficher les sous-tâches sous chaque tâche principale.

```

1 function renderTasks() {
2     const taskList = document.getElementById('taskList');
3     taskList.innerHTML = '';
4
5     tasks.forEach(task => {
6         const taskItem = document.createElement('div');
7         taskItem.className = 'task-item';
8
9         // Affichage de la tâche principale
10        taskItem.innerHTML = `
11            <span>${task.title}</span>
12            <button onclick="addSubtask(${task.id})">Ajouter une sous-
13            tâche</button>
14        `;
15
16        // Liste des sous-tâches
17        const subtaskList = document.createElement('ul');
18        task.subtasks.forEach(subtask => {
19            const subtaskItem = document.createElement('li');
20            subtaskItem.style.textDecoration = subtask.completed ? 'line-through' : 'none';
21            subtaskItem.innerHTML = `
22                <input type="checkbox" ${subtask.completed ? 'checked' : ''}
23                onclick="toggleSubtaskCompletion(${task.id},
24                ${subtask.id})">

```



```

23             ${subtask.title}
24             <button onclick="deleteSubtask(${task.id},
25             ${subtask.id})">Supprimer</button>
26             `;
27             subtaskList.appendChild(subtaskItem);
28         });
29         taskItem.appendChild(subtaskList);
30         taskList.appendChild(taskItem);
31     });
32 }

```

9. Export des tâches

- Ajoutez un bouton pour exporter la liste des tâches au format JSON ou CSV.
- Fournissez un fichier téléchargeable contenant les données des tâches.

Ajout d'un bouton d'exportation

Ajouter un bouton permettant de choisir le format d'exportation.

```

1 <div>
2     <button onclick="exportTasks('json')">Exporter en JSON</button>
3     <button onclick="exportTasks('csv')">Exporter en CSV</button>
4 </div>

```

Fonction pour exporter les tâches au format JSON

Cette fonction convertit les données des tâches en chaîne JSON et génère un fichier téléchargeable.

```

1 function exportTasks(format) {
2     if (format === 'json') {
3         const jsonData = JSON.stringify(tasks, null, 2);
4         downloadFile('tasks.json', jsonData, 'application/json');
5     } else if (format === 'csv') {
6         const csvData = convertTasksToCSV(tasks);
7         downloadFile('tasks.csv', csvData, 'text/csv');
8     }
9 }

```

Génération et téléchargement du fichier

Cette fonction crée un fichier à partir des données et déclenche son téléchargement.

```

1 function downloadFile(filename, content, mimeType) {
2     const blob = new Blob([content], { type: mimeType });
3     const link = document.createElement('a');
4     link.href = URL.createObjectURL(blob);
5     link.download = filename;
6     link.click();
7     URL.revokeObjectURL(link.href); // Libérer la mémoire
8 }

```

Conversion des tâches au format CSV

Créer une fonction pour convertir les données des tâches en format CSV.

```

1 function convertTasksToCSV(tasks) {
2     const headers = ['ID', 'Titre', 'Terminé', 'Sous-tâches'];
3     const rows = tasks.map(task => {
4         const subtasks = task.subtasks
5             .map(subtask => `${subtask.title} (${subtask.completed ?
6             'Terminé' : 'En cours'})`);

```

```

6         .join('; ');
7         return `${task.id},${task.title},${task.completed ? 'Oui' :
'Non'},"${subtasks}"`;
8     });
9
10    return [headers.join(', '), ...rows].join('\n');
11 }

```

10. Notifications

- Ajoutez une option pour configurer un rappel sur une tâche (par exemple, avec une date/heure).
- Utilisez les API de notification de navigateur pour envoyer des rappels aux utilisateurs.

Ajouter une interface utilisateur pour configurer un rappel

Inclure une option pour définir une date et une heure de rappel.

```

1 <div id="taskList">
2     <!-- Exemple pour une tâche -->
3     <div class="task-item">
4         <span>Tâche principale 1</span>
5         <label for="reminder">Rappel :</label>
6         <input type="datetime-local" id="reminder-1" />
7         <button onclick="setReminder(1)">Configurer</button>
8     </div>
9 </div>

```

Stocker les rappels pour chaque tâche

Ajouter une propriété **reminder** à chaque tâche pour enregistrer la date/heure du rappel.

```

1 let tasks = [
2     {
3         id: 1,
4         title: "Tâche principale 1",
5         completed: false,
6         reminder: null, // Date/heure du rappel
7         subtasks: []
8     },
9     {
10        id: 2,
11        title: "Tâche principale 2",
12        completed: false,
13        reminder: null,
14        subtasks: []
15    }
16 ];

```

Implémenter la logique pour configurer un rappel

Créer une fonction permettant de configurer un rappel et d'enregistrer la date dans l'objet **tasks**.

```
1 function setReminder(taskId) {
2     const task = tasks.find(t => t.id === taskId);
3     if (!task) return;
4
5     const reminderInput = document.getElementById(`reminder-${taskId}`);
6     const reminderTime = new Date(reminderInput.value);
7
8     if (isNaN(reminderTime.getTime())) {
9         alert("Veuillez entrer une date/heure valide.");
10        return;
11    }
12
13    task.reminder = reminderTime;
14    alert(`Rappel configuré pour ${task.title} à
15    ${reminderTime.toLocaleString()}`);
16    scheduleNotification(taskId, reminderTime);
17 }
```

Planifier les notifications

Utiliser **setTimeout** pour déclencher une notification au moment configuré.

```
1 function scheduleNotification(taskId, reminderTime) {
2     const now = new Date();
3
4     // Calculer le délai avant la notification
5     const delay = reminderTime.getTime() - now.getTime();
6     if (delay <= 0) {
7         alert("La date/heure doit être dans le futur.");
8         return;
9     }
10
11    setTimeout(() => {
12        const task = tasks.find(t => t.id === taskId);
13        if (!task) return;
14
15        showNotification(`Rappel pour la tâche : ${task.title}`);
16    }, delay);
17 }
```

Utiliser les API de notification du navigateur

Vérifier les permissions et afficher une notification.

```
1 function showNotification(message) {
2     if (!("Notification" in window)) {
3         alert("Les notifications ne sont pas supportées par ce
navigateur.");
4         return;
5     }
6
7     if (Notification.permission === "granted") {
```

```

8         new Notification(message);
9     } else if (Notification.permission !== "denied") {
10         Notification.requestPermission().then(permission => {
11             if (permission === "granted") {
12                 new Notification(message);
13             }
14         });
15     }
16 }

```

Activer les permissions de notification

Lorsque la page se charge, demander la permission pour les notifications si ce n'est pas déjà fait.

```

1 document.addEventListener("DOMContentLoaded", () => {
2     if (Notification.permission !== "granted" && Notification.permission
3     !== "denied") {
4         Notification.requestPermission();
5     }
6 });

```